Bounding a trained network's error by compressing it, without validation data

Claude (Anthropic)*

October 2026

Abstract

If a trained network f agrees on most inputs with a network g that has a short description, then an Occam bound on g plus the disagreement $\Pr[f \neq g]$ bounds the test error of f, with no validation data. We study when the disagreement term can be certified. When the input distribution is known, as for Boolean tasks with uniform inputs, it is free. A 289,281-parameter MLP then compresses into a 28–100-bit truth table, threshold function or small quantized network, and the bound on f is within 0.005 of its test error at $m = 16000$ for every noise-free target. At smaller m the gap is mostly the Occam term, of order $L \ln 2/m$. The bound stays informative for networks that have only partly learned their target (0.30 against a test error of 0.29 for a 3-parity, 0.34 against 0.17 for a 6-junta) and for networks that interpolate 10% label noise (0.17 against 0.14), where parameter-counting and VC bounds are vacuous. On 20-bit inputs, once f has learned the target, the compressed networks agree with it on all but at most a few of the 2^{20} inputs, which gives bounds valid for every input distribution. With only the training set, we prove a dithered transfer theorem. Compressing f by subtractive dithered quantization with public randomness makes $g - f$ independent noise. A certificate computed from g alone at each training point then transfers through the same Occam bound, giving a deterministic bound on f itself. This removes the $\sqrt{d}$ loss of worst-case perturbation sets. We prove the required per-input certificate for one-hidden-layer ReLU networks. On MNIST the bound is non-vacuous for narrow networks and for networks with sparse updates, and vacuous for wide dense ones, because it needs the weights described to the precision the margins require. The random initialization costs nothing, since codes may depend on data-independent randomness. We measure how coding relative to the initialization, the initialization scale and a deterministic initialization change the description length, and give an init-kernel codec. On lazily trained Boolean networks its best rank is the dimension of the degree-≤ 1 Walsh space for majority targets, which suggests, heuristically, a cost growing like the n^k sample complexity of kernel methods.

1 Introduction

A standard way to certify that a trained network generalizes is to hold out labelled data. Without held-out data, the classical tools (VC dimension, parameter counting, norm-based bounds) are vacuous for networks with more parameters than training examples. They bound every network in a huge class, not the one that was trained.

*Written by an AI system in a session prompted by Kaarel Hänni; not reviewed by a human. The proofs were checked by adversarial AI referees (Appendix B). Code, results and referee reports: https://github.com/kaarelh/AI-works, folder `compression-generalization/`.

This paper studies a reduction that targets the trained network itself. Suppose that some network g with a *short description* (a code of L bits fixed before seeing the data) agrees with the trained network f on most inputs. Then

$$\mathrm{err}(f) \;\le\; \mathrm{err}(g) + \Pr_x[f(x) \neq g(x)],$$

and since g ranges over at most 2^L descriptions, a Chernoff–Occam union bound over all of them bounds $\mathrm{err}(g)$ by its training error plus a term that is about $L \ln 2/m$ when g has no training errors (and of order $\sqrt{L/m}$ otherwise). Bounding generalization reduces to compressing the network, provided one can certify the closeness term $\Pr[f \neq g]$. That term concerns the input distribution, which is exactly what the training data does not reveal without depending on f.

We consider three regimes, in decreasing order of what is known about the input distribution, and ask three questions. Has this been done? Can a compression scheme give interesting bounds on real networks, beating parameter counting? Does the random initialization have to be described, which would make the description long?

Answers in brief (Table 1).

- *Prior work.* Compression bounds for the compressed network g are well developed [1, 26, 43]. The reduction to the original f with the closeness term estimated on *unlabeled* data goes back to Kääriäinen [20] and was recently carried out for modern networks by Bazinet et al. [3], with non-vacuous bounds on MNIST and CIFAR-10. With no extra data, the deterministic derandomization bounds of Neyshabur et al. [33] and Nagarajan and Kolter [31] bound the original network but are vacuous. The exception we know of is Biggs and Guedj [5], who obtain non-vacuous training-data-only bounds for one-hidden-layer networks with erf or GELU activations, via an exact identity between such a network and a stochastic majority vote.

- *Known input distribution* (Sections 5.1 and 5.2). On Boolean tasks with uniform inputs, $\Pr[f \neq g]$ is free. A 289,281-parameter MLP trained on $n = 50$ bits compresses into a 28–100-bit truth table, linear threshold function or small quantized network. The resulting bound on f is within 0.005 of its test error at $m = 16000$ for the noise-free targets. At smaller m the Occam term makes it looser, but it stays informative: 0.30 against 0.29 for a partially trained parity network and 0.34 against 0.17 for a partially trained 6-junta. For a network that interpolates 10% label noise, the bound at $m = 16000$ is 0.17 against 0.14. Parameter-counting and VC bounds are 1.0 throughout. For $n = 20$, once f has learned the target, the compressed networks agree with f on all but at most 8 of the 2^{20} inputs. Patching those exceptions gives bounds that hold for *every* input distribution (0.0065 for `parity3` at $m = 4000$).

- *Training data only* (Sections 3.1 and 5.4). We prove a *dithered transfer theorem.* Compress f by subtractive dithered quantization with public randomness, so that $g - f$ is independent uniform noise in weight space; compute from g alone a per-input certificate that f and g classify alike; and transfer the fraction of certified training points to the population with the same Occam bound. The result is a deterministic bound on the original f (the stored weights, evaluated in float64) with no validation or unlabeled data. We prove the per-input certificate for one-hidden-layer ReLU networks. The bound is non-vacuous for narrow one-hidden-layer MNIST networks: 0.44 for 8 hidden units, where the VC bound with d equal to the number of parameters gives 0.71. It is also non-vacuous for standard-width networks whose first-layer update is sparse in a DCT basis: 0.31 for 100 hidden units with 1% nonzero update coefficients, where coding the network exactly gives 0.62 (Table 6). It becomes vacuous for wide dense networks. The certificate needs f's weights described to the precision its margins require,

task	network f	extra data	test err. f	bound on f	bits	32-bit code	VC
`parity3` ($n = 50$, $m = 4000$)	MLP, 289k params	known D_X	0.0000	0.007	35	1.00	1.00
`parity3`, partly learned ($n = 50$, $m = 2000$)	MLP, 289k params	known D_X	0.2876	0.304	35	1.00	1.00
`junta6` ($n = 50$, $m = 4000$)	MLP, 289k params	known D_X	0.0223	0.042	102	1.00	1.00
`maj5`, 10% label noise ($n = 50$, $m = 16000$)	MLP, 289k params	known D_X	0.1390	0.174	53	1.00	1.00
`parity3` ($n = 20$, $m = 4000$)	MLP, 274k params	none (any D)	0.0000	0.006	33	1.00	1.00
MNIST	784-100-10	unlabeled inputs	0.0229	0.313	18,371	1.00	1.00
Fashion-MNIST	784-100-10	unlabeled inputs	0.1168	0.532	16,594	1.00	1.00
MNIST	784-8-10	none	0.0697	0.439	23,490	0.94	0.71
MNIST	784-16-10	none	0.0512	0.553	43,658	0.99	0.87
MNIST	784-100-10, 1% update	none	0.0462	0.314	17,664	0.62	–
MNIST	784-256-10, 0.3% update	none	0.0441	0.399	23,244	0.79	–

Table 1: Headline results: certified upper bounds on the test error of the trained network itself, at confidence 95%, against its measured test error and two parameter-counting baselines for the same network: the Occam bound for coding it exactly with 32 bits per parameter (for sparse-update networks: per stored parameter, plus the support), and the VC bound with d equal to the number of parameters (optimistic). 'Extra data': what the bound uses besides the training set; no row uses labelled held-out data. Chance level is 0.5 for the Boolean tasks and 0.9 for MNIST. Boolean rows average three seeds; image rows are one network each, with $m = 60000$. 'Bits': description length of the compressed or dithered network.

and that costs bits in proportion to the width. Worst-case versions of the same idea fail by a factor $\sqrt{d}$ (Theorem 4).

- *Initialization* (Sections 3.2 and 5.5). The initialization costs nothing: any data-independent randomness may be used by the code (Lemma 8). Function-space codecs never refer to it, and weight-space codecs code the update relative to it. We measure how coding relative to the initialization or to zero, the initialization scale and a deterministic initialization affect the description length. We give an init-kernel codec (Proposition 9). On lazily trained Boolean networks with majority targets its best rank is $1 + n$, the dimension of the degree-≤ 1 Walsh space; this suggests, as a heuristic we have not tested across n, that its cost grows like the n^k sample complexity of kernel methods for degree-k targets. Lazily and richly trained Boolean networks distil into the same short juntas, so the training regime matters for weight-space codecs, not for function-space ones.

Scope. Everything here is a theorem or a reproducible measurement on small networks: MLPs on 20–50-bit Boolean tasks, MNIST and Fashion-MNIST, on a 4-core CPU. Every bound is computed with explicit code lengths, confidence parameters and a charge for the design choices we made on the same data (Section 5). The code, the result files and an AI referee record are in the repository.

2 Related work

Occam, MDL and PAC-Bayes. Bounding the error of a hypothesis by its description length is the Occam bound [7, 21]. Applied to network weights it is the minimum-description-length programme of Hinton and van Camp [16] and the flat-minima argument of Hochreiter and Schmidhuber [17]. PAC-Bayes [28] lets the hypothesis be a distribution over weights; Langford and Caruana [22] and Dziugaite and Roy [11] obtained non-vacuous PAC-Bayes bounds for stochastic networks, the latter with a prior centred at the random initialization, and Pérez-Ortiz et al. [34] and Dziugaite et al. [12] tightened them with data-dependent priors.

Bounds for compressed networks. Arora et al. [1] bound a compressed network g obtained by random projections from a trained f. Zhou et al. [43] combine pruning, quantization and coding with a noisy posterior and obtain non-vacuous bounds at ImageNet scale for the noisy compressed network. Lotfi et al. [26] train in a random subspace [24], quantize and arithmetic-code, and obtain the tightest data-independent bounds we know of (11.6% on MNIST and 32.8% on Fashion-MNIST, values from their repository); Lotfi et al. [27] extend this to language models. In all of these the certified predictor is the compressed (or noisy, or subspace-trained) network, not an independently trained f.

Bounds for the original network through a surrogate. The reduction $\mathrm{err}(f) \le \mathrm{err}(g) + \Pr[f \neq g]$ with the disagreement estimated on unlabeled data goes back at least to Kääriäinen [20]. Hsu et al. [18] use a distilled network and control the distance with samples from an augmentation distribution. Garg et al. [13] certify the original network using randomly labelled unlabeled data. Most directly, Bazinet et al. [3] implement the reduction for modern networks with surrogates certified by sample compression, PAC-Bayes or Lotfi-style compression, and disagreement on a held-out unlabeled set. Their repository reports bounds on the unmodified network of 1.78–3.45% for an MNIST CNN, depending on the surrogate, and 17–35% for a CIFAR-10 ResNet-18. Our unlabeled-data experiments are an instance of their method. Theoretical treatments of the same step without extra data include Suzuki et al. [37] (local Rademacher complexity of the difference class), Daniely and Granot [10] (approximation at every input), Sefidgaran et al. [36] (rate-distortion) and the surrogate framework of Negrea et al. [32]; none yields numerical bounds.

Deterministic bounds from noise resilience. Derandomizing a PAC-Bayes bound through margins goes back to Langford and Shawe-Taylor [23]. Neyshabur et al. [33] derandomize PAC-Bayes for the original network using margins and robustness to weight perturbations; Nagarajan and Kolter [31] instead require per-input noise resilience (stability of the hidden activations and the output under perturbation) on the training points and transfer it to test points; Wei and Ma [39] use an all-layer margin. These bounds are vacuous in practice. Biggs and Guedj [5] obtain non-vacuous bounds from the training data for deterministic one-hidden-layer networks with erf or GELU activations. By our reading of their slides (unverified), they report about 0.69–0.77 on MNIST with data-free priors and 0.29–0.49 with priors trained on a split of the training set, which is still a training-data-only bound. Their bounds rest on the exact identity $\mathbb{E}[\mathrm{sign}(w \cdot x + \mathrm{noise})] \propto \mathrm{erf}(\cdot)$, which makes such a network a PAC-Bayes majority vote; see also Biggs and Guedj [4]. Our dithered transfer theorem shares the structure of these derandomization arguments (a perturbation set around f plus a per-point condition transferred from training to test). What differs is the mechanism: a single public random draw, an Occam code of the perturbed network, and a per-input concentration bound in place of worst-case or norm-based perturbation bounds. It applies to ReLU networks, for which no exact majority-vote identity is available; it does not give tighter bounds than Biggs and Guedj [5]. We claim novelty only for this single-draw, Occam-coded instance and its non-vacuous numbers. Sound bound propagation under weight perturbations [40] and subtractive dither [35] are the tools; shared-randomness model coding also appears in Havasi et al. [15], and single-draw arguments in Blanchard and Fleuret [6].

Boolean tasks and simplicity. Valle-Pérez et al. [38] bound the Bayesian posterior of networks on 7-bit Boolean functions with a prior induced by random initialization. Our Boolean experiments instead bound the network produced by gradient descent, via distillation into a junta, a linear threshold function or a small quantized network, with the disagreement computed exactly or on free

samples from the known input distribution. A companion project in the same repository [9] reports similar certificates for rich-regime networks on Boolean targets, using unlabelled inputs. When the labelling function is also known, a network's error can be computed outright; Gross et al. [14] study proofs of such statements and the trade-off between proof length and tightness.

Initialization. A data-independent initialization costs nothing in PAC-Bayes [11] and in Occam codes [1, 26]. Supermasks [42] show that a random initialization can carry most of the information of a good network at zero cost. Lazy and rich training [8, 19, 29] predict how the update $\theta - \theta_0$ is structured, and deterministic initializations exist [41]. We are not aware of a study measuring the description length of the update as a function of the initialization scale, or comparing coding relative to the initialization and relative to zero.

3 From a short description to a bound on the trained network

Setting. D is a distribution on $\mathcal{X} \times \mathcal{Y}$ with input marginal D_X, and the training sample is $S = ((x_i, y_i))_{i \le m} \sim D^m$. For a predictor h, $\mathrm{err}(h) = \Pr_D[h(x) \neq y]$ and $k_S(h) = \#\{i : h(x_i) \neq y_i\}$. The trained network $f = f_S$ is an arbitrary function of S and of any other randomness used in training. A *code* is a countable set Σ of descriptions with lengths $L : \Sigma \to [0, \infty)$ satisfying $\sum_\sigma 2^{-L(\sigma)} \le 1$, fixed before S is seen; σ decodes to a predictor g_σ. All our lengths are $-\log_2$ of products of conditional probabilities (Elias-δ codes for integers, enumerative codes for subsets, adaptive Krichevsky–Trofimov codes for integer arrays), so they need not be integers. We write $\overline{\mathrm{Bin}}(k, m, \alpha) = \sup\{p : \Pr[\mathrm{Bin}(m, p) \le k] \ge \alpha\}$ for the binomial tail inversion.

Lemma 1 (Occam bound, exact binomial form)**.** *With probability at least $1-\delta$ over S, simultaneously for all $\sigma \in \Sigma$,*

$$\mathrm{err}(g_\sigma) \le \overline{\mathrm{Bin}}\big(k_S(g_\sigma), m, \delta 2^{-L(\sigma)}\big) \le \mathrm{kl}^{-1}\Big(\tfrac{k_S(g_\sigma)}{m},\ \tfrac{L(\sigma)\ln 2 + \ln(1/\delta)}{m}\Big).$$

Proof. For fixed σ, $k_S(g_\sigma) \sim \mathrm{Bin}(m, \mathrm{err}(g_\sigma))$, so $\mathrm{err}(g_\sigma) > \overline{\mathrm{Bin}}(k_S, m, \alpha)$ says that k_S falls in a lower tail of mass less than $\alpha = \delta 2^{-L(\sigma)}$. Sum over σ using Kraft. The second inequality is the Chernoff bound $\Pr[\mathrm{Bin}(m, p) \le qm] \le e^{-m\,\mathrm{kl}(q\|p)}$ for $q \le p$. □

The description may be chosen after seeing S in any way.

Theorem 2 (reduction to compression)**.** *For any predictor f and any description $\sigma = \sigma(S)$, $\mathrm{err}(f) \le \mathrm{err}(g_\sigma) + \Pr_{D_X}[f(x) \neq g_\sigma(x)]$. If, after σ is fixed, N inputs are drawn i.i.d. from D_X independently of everything else and $f \neq g_\sigma$ on j of them, then with probability at least $1 - \delta_1 - \delta_2$,*

$$\mathrm{err}(f) \le \overline{\mathrm{Bin}}\big(k_S(g_\sigma), m, \delta_1 2^{-L(\sigma)}\big) + \overline{\mathrm{Bin}}(j, N, \delta_2).$$

Proof. $\{f(x) \neq y\} \subseteq \{g(x) \neq y\} \cup \{f(x) \neq g(x)\}$. The first term is Lemma 1; conditional on (S, σ) the disagreement indicators are i.i.d. Bernoulli with mean $\Pr_{D_X}[f \neq g_\sigma]$, so Clopper–Pearson gives the second. □

The disagreement term needs *inputs* from D_X but no labels. We use three sources: (i) a known, sampleable D_X (Boolean tasks with uniform inputs: the inputs are free, and for $\{\pm 1\}^n$ with $n \le 22$ we enumerate them, making the term exact); (ii) unlabeled samples (MNIST test images without labels; this is the setting of Bazinet et al. [3]); (iii) none at all (Section 3.1). If K candidate surrogates are compared on the same N inputs, δ_2 is replaced by δ_2/K; our experiments either select on a separate pool or apply this correction.

Corollary 3 (distribution-free, finite domain)**.** *Let $\mathcal{X}$ be finite. If a description g' satisfies $g'(x) = f(x)$ for all $x \in \mathcal{X}$, then for every distribution D, with probability $\geq 1-\delta$, $\mathrm{err}_D(f) \leq \overline{\mathrm{Bin}}(k_S(f), m, \delta 2^{-L(g')})$. A short g that disagrees with f on a set E becomes such a g' by appending E, at a cost of $1 + L_\delta(|E|+1) + \log_2 \binom{|\mathcal{X}|}{|E|}$ bits.*

3.1 Bounds from training data alone

The disagreement term is a statement about D_X. To avoid it, make the closeness certificate a function of the short description only, evaluate it at the training inputs, and transfer it with the same Occam bound.

Theorem 4 (certified transfer)**.** *Let each σ define a set K_σ of predictors and a measurable $\pi_\sigma : \mathcal{X} \times \mathcal{Y} \to \{0,1\}$ with $\pi_\sigma(x,y) = 1 \Rightarrow h(x) = y$ for all $h \in K_\sigma$. With probability $\geq 1-\delta$, simultaneously for all σ and all $h \in K_\sigma$,*

$$\mathrm{err}(h) \leq \overline{\mathrm{Bin}}(\#\{i : \pi_\sigma(x_i, y_i) = 0\}, m, \delta 2^{-L(\sigma)}).$$

Proof. $\mathrm{err}(h) \leq \Pr_D[\pi_\sigma = 0]$ for $h \in K_\sigma$; apply Lemma 1 to the losses $1 - \pi_\sigma$. □

For networks, K_σ is a set of weights around a quantized network and π_σ a sound interval bound; $f \in K_\sigma$ is checked on the weights. This works poorly for dense networks. If the residual $f - g$ is spread over d coordinates, the worst case of $|\Delta w \cdot x|$ over a box or ball containing it (any set that is cheap to describe without knowing the residual's direction relative to x) is larger than its actual value by a factor of order $\sqrt{d}$. On a 784–100–10 MNIST network the ratio of certified to actual first-layer error is 12–13 at every step size we tried, and certifying 95% of the training points needs about 8 bits per first-layer coefficient (Section 5.4). Randomizing the quantization removes the $\sqrt{d}$.

Dithered compression. Let ξ be public randomness drawn independently of S, of the test point and of any auxiliary randomness ζ used to tune the encoder. In our code ξ is generated from fresh operating-system entropy after training and after tuning, and the pseudo-random generator is modelled as truly random, as for any randomized bound. An encoder maps (f, ξ) to a description $\sigma = \sigma_{S,\zeta}(f, \xi)$; it may depend on S and ζ arbitrarily, for instance through step sizes chosen on the training data. Each (σ, ξ) defines a certificate $\pi_{\sigma,\xi}$.

Theorem 5 (dithered transfer)**.** *Suppose that for every fixed encoder (every value of (S, ζ)), every network f and every (x, y) there is an event $\mathrm{Bad}_{x,y}(f, \xi)$ with $\Pr_\xi[\mathrm{Bad}_{x,y}] \leq \eta$ such that $\pi_{\sigma(f,\xi),\xi}(x, y) = 1$ and not $\mathrm{Bad}_{x,y}$ imply $f(x) = y$. Then with probability $\geq 1 - \delta - \delta'$ over (S, ζ, ξ),*

$$\mathrm{err}(f) \leq \overline{\mathrm{Bin}}(\#\{i : \pi(x_i, y_i) = 0\}, m, \delta 2^{-L(\sigma)}) + \eta/\delta'.$$

Proof. $\mathrm{err}(f) \leq \Pr_D[\pi = 0] + \Pr_D[\mathrm{Bad}]$. Conditional on ξ, S is i.i.d. from D and $\{\pi_{\sigma,\xi}\}_\sigma$ is a fixed family, so Lemma 1 bounds the first term for all σ at once. Conditional on (S, ζ), which fixes f and the encoder, Fubini gives $\mathbb{E}_\xi \Pr_D[\mathrm{Bad}] = \mathbb{E}_D \Pr_\xi[\mathrm{Bad}] \leq \eta$, and Markov's inequality gives $\Pr_\xi[\Pr_D[\mathrm{Bad}] > \eta/\delta'] \leq \delta'$. □

Hyperparameters chosen by looking at ξ itself would need a union bound in the Markov step; we choose them with an independent selection dither ζ and certify with a fresh ξ. Hyperparameters that change the certificate (step sizes, the basis, t_1, t_2) must be part of σ or fixed in advance, so that the Occam step covers them.

Lemma 6 (subtractive dither [35])**.** *For $a \in \mathbb{R}$, $s > 0$ and $U \sim \mathrm{Unif}[-\frac{1}{2}, \frac{1}{2}]$, the error $s(\mathrm{round}(a/s + U) - U) - a$ is uniform on $[-\frac{s}{2}, \frac{s}{2}]$ whatever a is. With independent U's per coordinate, the errors are independent, the error in a coordinate with step s_k is uniform on $[-\frac{s_k}{2}, \frac{s_k}{2}]$, and their joint law does not depend on f.*

The decoder knows ξ and the integers $K = \mathrm{round}(a/s + \xi)$, so g costs $L(K)$ bits while $g - f$ is noise of known law. The next lemma turns this into a per-input certificate for one-hidden-layer networks.

Lemma 7 (certificate for one-hidden-layer networks)**.** *Let $f(x) = V\mathrm{relu}(Wx + b) + c$ with $W = W_0 + AB^\top$ (W_0 the initialization, B an orthonormal basis), $V \in \mathbb{R}^{C\times H}$, and let g be obtained by subtractive dithering of A (steps $s_{ik} \geq 0$; a step of 0 means the coefficient is sent exactly, as we do for coefficients that are exactly zero), $b - b_0$ (step s^b), $V - V_0$ (s^V) and $c - c_0$ (s^c). Fix (x, y), put $\hat{x} = B^\top x$, $v_i = \sum_k s_{ik}^2 \hat{x}_k^2/12 + (s^b)^2/12$ and $\tau_i = t_1\sqrt{v_i}$, and let u_i', z' be g's pre-activations and logits. For $c \neq y$ let $D_i = |V'_{yi} - V'_{ci}| + s^V$, let w_i be 1 if $u_i' \geq 2\tau_i$, 0 if $u_i' \leq -2\tau_i$ and 2 otherwise, and*

$$\hat{P}_c = \textstyle\sum_i w_i D_i^2 v_i + \frac{(s^V)^2}{6}\sum_i \mathrm{relu}(u_i' + 2\tau_i)^2 + \frac{(s^c)^2}{6}, \qquad \hat{B}_c = \textstyle\sum_{i:|u_i'|<2\tau_i} D_i\hat{\beta}_i,$$
$$\hat{\beta}_i = \min\Big(\tfrac{\sqrt{v_i}}{2},\ \sqrt{2\pi v_i}\,\bar{\Phi}\big((|u_i'| - \tau_i)_+/\sqrt{v_i}\big)\Big).$$

Let $\pi_g(x, y) = 1$ iff $z'_y - z'_c > t_2\sqrt{2\hat{P}_c} + \hat{B}_c$ for every $c \neq y$. Then the hypothesis of Theorem 5 holds with $\eta = 2He^{-t_1^2/2} + 2(C-1)e^{-t_2^2}$. (Binary networks with one logit: $D_i = |V_i'| + s^V/2$, the last two terms of $\hat{P}$ carry $1/12$, and $C - 1$ is replaced by 1.)

Proof. Let ϵ, e, Ξ, γ be the dither errors of A, b, V, c, $\nu_i = \sum_k \epsilon_{ik}\hat{x}_k + e_i$, and u_i the pre-activations of f, so $u_i' = u_i + \nu_i$.

(a) Tails. If $U \sim \mathrm{Unif}[-a, a]$ then $\mathbb{E}e^{\lambda U} = \sinh(\lambda a)/(\lambda a) \leq e^{\lambda^2 a^2/6}$, so U is sub-Gaussian with variance proxy $a^2/3$, its variance. Hence the ν_i are independent, symmetric and sub-Gaussian with proxies v_i, and $G = \{|\nu_i| \leq \tau_i\ \forall i\}$ has $\Pr[G^c] \leq 2He^{-t_1^2/2}$.

(b) Decomposition. Fix $c \neq y$; let $d_i = V_{yi} - V_{ci}$ and $\chi_i = \Xi_{yi} - \Xi_{ci}$. The margin error is $F = \sum_i d_i(\mathrm{relu}(u_i + \nu_i) - \mathrm{relu}(u_i)) + \sum_i \chi_i \mathrm{relu}(u_i + \nu_i) + \gamma_y - \gamma_c$. Replacing each ν_i by $\mathrm{clip}_{\tau_i}(\nu_i)$ gives $\tilde{F}$, and $F = \tilde{F}$ on G. With $\varphi_i(t) = \mathrm{relu}(u_i + \mathrm{clip}_{\tau_i}(t)) - \mathrm{relu}(u_i)$: if $u_i \geq \tau_i$ then $\varphi_i = \mathrm{clip}_{\tau_i}$ (*active*); if $u_i \leq -\tau_i$ then $\varphi_i \equiv 0$ (*inactive*); otherwise φ_i is 1-Lipschitz (*marginal*).

(c) Moment generating function. Conditional on $(\nu_i)_i$, the Ξ-term is a centred linear form in independent uniforms with proxy at most $\frac{(s^V)^2}{6}\sum_i \mathrm{relu}(u_i + \tau_i)^2$, and $\gamma_y - \gamma_c$ has proxy $(s^c)^2/6$. For an active unit, $\mathrm{clip}(\nu_i)$ is symmetric with $|\mathrm{clip}(\nu_i)| \leq |\nu_i|$, so $\mathbb{E}e^{\lambda d_i \mathrm{clip}(\nu_i)} = \mathbb{E}\cosh(\lambda d_i\,\mathrm{clip}(\nu_i)) \leq \mathbb{E}\cosh(\lambda d_i\nu_i) \leq e^{\lambda^2 d_i^2 v_i/2}$. For a marginal unit, let X' be an independent copy of $X = \nu_i$; by Jensen, $\mathbb{E}e^{\lambda(\varphi(X) - \mathbb{E}\varphi(X))} \leq \mathbb{E}e^{\lambda(\varphi(X) - \varphi(X'))} = \mathbb{E}\cosh(\lambda(\varphi(X) - \varphi(X'))) \leq \mathbb{E}\cosh(\lambda(X - X')) \leq e^{\lambda^2 \cdot 2v_i/2}$, a proxy of $2d_i^2 v_i$. Multiplying the independent factors, $\tilde{F} - \mathbb{E}\tilde{F}$ is sub-Gaussian with proxy $P = \sum_{\mathrm{act}} d_i^2 v_i + 2\sum_{\mathrm{marg}} d_i^2 v_i + \frac{(s^V)^2}{6}\sum_i \mathrm{relu}(u_i + \tau_i)^2 + \frac{(s^c)^2}{6}$, and $\Pr[|\tilde{F} - \mathbb{E}\tilde{F}| \geq t_2\sqrt{2P}] \leq 2e^{-t_2^2}$.

(d) Bias. Only marginal units contribute to $\mathbb{E}\tilde{F}$. For the symmetric variable $\tilde{\nu} = \mathrm{clip}(\nu)$, $0 \leq \mathbb{E}\mathrm{relu}(u + \tilde{\nu}) - \mathrm{relu}(u) = \mathbb{E}(\tilde{\nu} - |u|)_+$, which is at most $\mathbb{E}\tilde{\nu}_+ = \mathbb{E}|\tilde{\nu}|/2 \leq \sqrt{v}/2$ and at most $\mathbb{E}(\nu - |u|)_+ = \int_{|u|}^\infty \Pr[\nu > t]\,dt \leq \sqrt{2\pi v}\,\bar{\Phi}(|u|/\sqrt{v})$.

(e) Computing from g. On G, $|u_i - u_i'| \leq \tau_i$: so $u_i' \geq 2\tau_i$ implies active, $u_i' \leq -2\tau_i$ implies inactive, $|u_i| \geq (|u_i'| - \tau_i)_+$ and $\mathrm{relu}(u_i + \tau_i) \leq \mathrm{relu}(u_i' + 2\tau_i)$; always $|d_i| \leq |V'_{yi} - V'_{ci}| + s^V$. Hence on G, $P \leq \hat{P}_c$ and $|\mathbb{E}\tilde{F}| \leq \hat{B}_c$.

(f) Conclusion. Let $\mathrm{Bad} = G^c \cup \bigcup_{c\neq y}\{|\tilde{F}_c - \mathbb{E}\tilde{F}_c| \geq t_2\sqrt{2P_c}\}$, so $\Pr[\mathrm{Bad}] \leq \eta$. Off Bad, $|F_c| < t_2\sqrt{2\hat{P}_c} + \hat{B}_c$ for all $c \neq y$, so $\pi_g(x, y) = 1$ forces $z_y > z_c$ for all $c \neq y$, i.e. $f(x) = y$. □

The certificate depends only on (K, steps, ξ) and (x, y), as Theorem 5 requires. The constants are what make the bound typical-case: the threshold scales as $t_2\sqrt{2\sum_i D_i^2 v_i}$, an ℓ_2 aggregate of the per-coordinate noise, instead of the ℓ_1 aggregate $\sum_i D_i \sum_k s_{ik}|\hat{x}_k|/2$ of a worst-case set.

Remark 1 (relation to PAC-Bayes). The law of $g = f + \epsilon$ is a posterior (uniform on a box around f), and $L(K)$ plays the role of its relative entropy to a prior. Theorem 5 uses one draw instead of an average over the posterior, pays η/δ' for it, and reaches the deterministic f through a per-input concentration statement. The margin-based derandomizations of Neyshabur et al. [33] and Nagarajan and Kolter [31] instead control the perturbation's effect uniformly over inputs, with norm-based bounds.

3.2 Initialization

Lemma 8 (the initialization is free). *Let θ_0 be drawn from any distribution independent of S, and let the code $(\Sigma_{\theta_0}, L_{\theta_0})$ depend on θ_0 (for instance, it describes $\theta - \theta_0$). Then Lemma 1 holds for it with probability $\geq 1 - \delta$ over (S, θ_0).*

Proof. Conditional on $\theta_0 = t$, $S \sim D^m$; apply Lemma 1 to the fixed code (Σ_t, L_t) and average over t. □

The same holds for every other piece of data-independent randomness: random projections, the dither, the DCT basis. A seed chosen as the best of N after training costs $\log_2 N$ bits; we charge 32 bits for the seed throughout, although with the seed fixed in advance even that is unnecessary. What the initialization can still cost is indirect: the description of the update $\theta - \theta_0$ depends on how training used θ_0.

Proposition 9 (init-kernel codec, linearized regime). *Let $J(x) = \partial_\theta f_{\theta_0}(x)$, $F_0 = \mathbb{E}_{D_X}[J(x)^\top J(x)]$, $f^{\text{lin}}_{\theta_0+\Delta}(x) = f_{\theta_0}(x) + J(x)\Delta$ and P_r the projector onto the top-r eigenvectors of F_0. Then*

$$\mathbb{E}_{D_X}\|f^{\text{lin}}_{\theta_0+\Delta}(x) - f^{\text{lin}}_{\theta_0+P_r\Delta}(x)\|^2 = \|F_0^{1/2}(I - P_r)\Delta\|^2 \leq \lambda_{r+1}(F_0)\|(I - P_r)\Delta\|^2.$$

When D_X is known, F_0 depends only on θ_0 and D_X, so its eigenbasis is free (Lemma 8) and $\theta_0 + P_r\Delta$ is described by r quantized coefficients. Heuristically, for uniform inputs on $\{\pm 1\}^n$ and an initialization distribution invariant under permutations and sign flips of the input coordinates (e.g. i.i.d. symmetric weights), the *expected* kernel is a function of $\langle x, x'\rangle$; its eigenfunctions are Walsh characters, and the degree-ℓ eigenspace has dimension $\binom{n}{\ell}$. The codec uses the empirical kernel of the one sampled θ_0, which for wide networks is close to the expected one. If, in addition, the eigenvalues decrease with the degree, a lazily trained network fitting a target of degree k (or the sign of one, as for majority functions and degree 1) needs $r \approx \sum_{\ell \leq k}\binom{n}{\ell}$ coefficients, so this codec certifies generalization once $m \gtrsim n^k$, the familiar sample complexity of kernel methods on degree-k targets. This is a heuristic: we have neither proved the eigenvalue ordering for our architecture nor varied n in experiments. A function-space codec instead describes any network that is close to a k-junta, however it was trained, with about $\log_2\binom{n}{k} + 2^k$ bits. The n^k cost belongs to the weight-space codec, not to the network.

4 Compression schemes

All code lengths are computed by `code/cgb/coding.py`: integers by Elias-δ (signed integers by the zigzag map), subsets of $[n]$ of size k by $L_\delta(k+1) + \log_2\binom{n}{k}$, and integer arrays of known shape by

the cheaper of two codes plus one flag bit. The dense code sends the range and then an adaptive Krichevsky–Trofimov (KT) code over it; the sparse code sends the support as a subset and then the nonzero values with a KT code. Step sizes lie on the grid $2^{e/4}$, $e \in \mathbb{Z}$, and are sent as signed integers. A unit test checks the Kraft inequality for each component (`code/tests/test_bounds_coding.py`).

Boolean tasks: function-space compression. Inputs are uniform on $\{\pm 1\}^n$ and the trained network f is an MLP of widths $(n, 512, 512, 1)$. The codec only queries f on inputs drawn from the uniform distribution, independently of the training set. (i) It estimates the influences $\mathrm{Inf}_j(f) = \Pr[f(x) \neq f(x^{\oplus j})]$ and proposes coordinate sets J, the coordinates with influence above τ for $\tau \in \{0.3, \ldots, 0.005\}$, plus $J = [n]$. (ii) For each J with $|J| \leq 14$ it builds the *truth table* $g(x) = T[x_J]$, where T is the majority of f over each fibre (estimated on a pool of 10^5 uniform inputs) and is sent with a KT code. (iii) For each J and hidden width $h \in \{0, 1, 2, 4, 8, 16\}$ ($h = 0$ is a linear threshold function) it distils a student MLP on x_J from f's labels on the pool. It then quantizes the student at three step sizes with quantization-aware fine-tuning (straight-through estimator) and sends its integers. (iv) It keeps the candidate minimizing the estimated bound: the Occam term plus the disagreement on a separate selection pool of 3×10^4 uniform inputs. (v) The certified disagreement is computed afterwards, either exactly on all 2^n inputs ($n = 20$) or on 2×10^5 fresh inputs ($n = 50$). The description includes a 2-bit codec-family tag, the subset J, the architecture and the steps. None of the codec's choices needs a union bound in the Occam term, because they are all part of the description.

Images: weight-space compression relative to the initialization. For an MLP f on 28×28 images, the compressed g keeps f's architecture and initialization θ_0 (free by Lemma 8; we still charge 32 bits for the seed). It codes the update $\theta - \theta_0$, with the first layer's update expressed in the orthonormal 2-D DCT basis and quantized with one step per frequency band $(u + v \in [0, 4), [4, 8), [8, 14), [14, 22), [22, 32), [32, \infty))$. The integers are coded per band. Starting from f, g is fine-tuned through the quantizer, with either training labels, f's soft labels on the training inputs (distillation), or both. The objective adds a rate penalty $\lambda \sum |c|/s$ that pushes coefficients to zero. Both this fine-tuning and the choice of (step, λ, loss) may use the training set freely.

Images and Boolean tasks: dithered compression for training-data-only bounds. For one-hidden-layer networks, g is the subtractive dither of f (Lemma 6) with steps $s_{ik} = 2^{(e_i + e'_{\mathrm{band}(k)})/4}$. The per-band exponents follow $(\mathrm{rms}\, \hat{x}_{\mathrm{band}})^{-p_b}$ and the per-unit exponents follow $\|V_{:,i}\|^{-p_u}$, both times a global scale. The steps are part of the description, and the integers are coded per band. The hidden biases and the output weights get a common step, a multiple $v_m \in \{1, 4, 16\}$ of 0.002 times the global scale, and the output bias the fixed step 2^{-10}. For networks whose first-layer update is exactly sparse (Section 5.4), the description carries a 1-bit flag and then, per frequency band, the support of the update as an enumerative subset code, before the values; coefficients off the support have step 0, so they are sent exactly and contribute no noise to the certificate. The global scale, the exponents (p_b, p_u) and v_m are chosen on a subsample of the training set with one dither draw; the bound is then computed with a fresh draw on the full training set. We use $t_1 = 5.8$ and $t_2 = 3.9$ throughout, fixed before the final runs and charged as one of four allowed pairs (2 bits); $\eta/\delta' \leq 2 \times 10^{-3}$ at $\delta' = 0.01$ for $H \leq 128$, and every reported bound uses the η of its own H. The description also carries 2 bits for the initialization scheme and the reference (initialization or zero).

Baselines. For every trained f we report the Occam bound with 32 bits per parameter (*parameter counting*; for sparse-update networks, 32 bits per stored parameter plus the same support code) and the VC bound $\hat{e} + \sqrt{(d(\ln(2m/d) + 1) + \ln(4/\delta))/m}$ with d equal to the number of parameters. The latter is optimistic for the baseline, since the VC dimension of ReLU networks with W weights and L layers is $O(WL \log W)$ and $\Omega(WL \log(W/L))$ [2].

Confidence. $\delta = 0.05$ throughout. In Theorem 2 it is split equally between the two terms, unless the disagreement is computed exactly. For the dithered bounds, $\delta = 0.04$ and $\delta' = 0.01$.

5 Experiments

All numbers in this section are produced by the scripts in `code/experiments` and are reported in the summary files of `results`; the tables below are generated from those files by `code/experiments/make_tables.py`. Every reported bound includes a charge for the codec designs we tried on the same data during development (4 bits per description, and the disagreement term at confidence $\delta_2/16$; `code/experiments/accounting.py`).

5.1 Boolean tasks with a known input distribution

Setup. Inputs are uniform on $\{\pm 1\}^{50}$. Targets: the parity of 3 fixed coordinates (`parity3`), the majority of 5 (`maj5`), the AND of 3 (`and3`), a random truth table on 6 coordinates (`junta6`), and the majority of all 50 (`fullmaj`), plus `maj5` and `parity3` with 10% of the training labels flipped. The network f is an MLP of widths $(50, 512, 512, 1)$ with 289,281 parameters, trained with Adam (learning rate 10^{-3}, batch 64) until it reaches zero training error and a mean training loss below 0.01. We use $m \in \{250, \ldots, 16000\}$ and three seeds per setting (training set and initialization). The bound is Theorem 2 with the codec of Section 4; the disagreement term uses 2×10^5 fresh uniform inputs.

Results. Figure 1 and Table 2 show the main finding: once m is large compared with the description length, the bound on f is close to its test error (within 0.005 at $m = 16000$ for every noise-free target). The gap is mostly the Occam term, of order $L \ln 2/m$ with L of 28–100 bits (a truth table on the relevant coordinates or a linear threshold function), so at small m the bound is several times the test error: 0.26 against 0.066 for `fullmaj` at $m = 500$, 0.09 against 0.011 for `maj5`. The parameter-counting baseline (32 bits per parameter, about 9.3×10^6 bits) and the VC baseline are 1.0 for every network. Where f has not learned the target (`parity3` at $m \leq 1000$), the bound is vacuous, correctly. Where f has partially learned it, the bound stays informative: at $m = 2000$, `parity3` networks have test error 0.29 and bound 0.30 (the best g is then the true parity, with zero training errors, and $\Pr[f \neq g]$ is f's error), while `junta6` networks have test error 0.17 and bound 0.34.

Interpolating noisy labels. With 10% label noise, f fits every flipped label (zero training error). Its test error on noisy labels is $0.1 + 0.8 \Pr[f \neq \text{target}]$. The compressed g is the clean rule, so $\text{err}(g) \approx 0.1$ and $\Pr[f \neq g]$ measures the damage done by memorization. At $m = 16000$ the bounds are 0.17 against a test error of 0.14 for both tasks. A bound of this kind needs only that f is close to a short program on most of the input space; it does not need f to be simple.

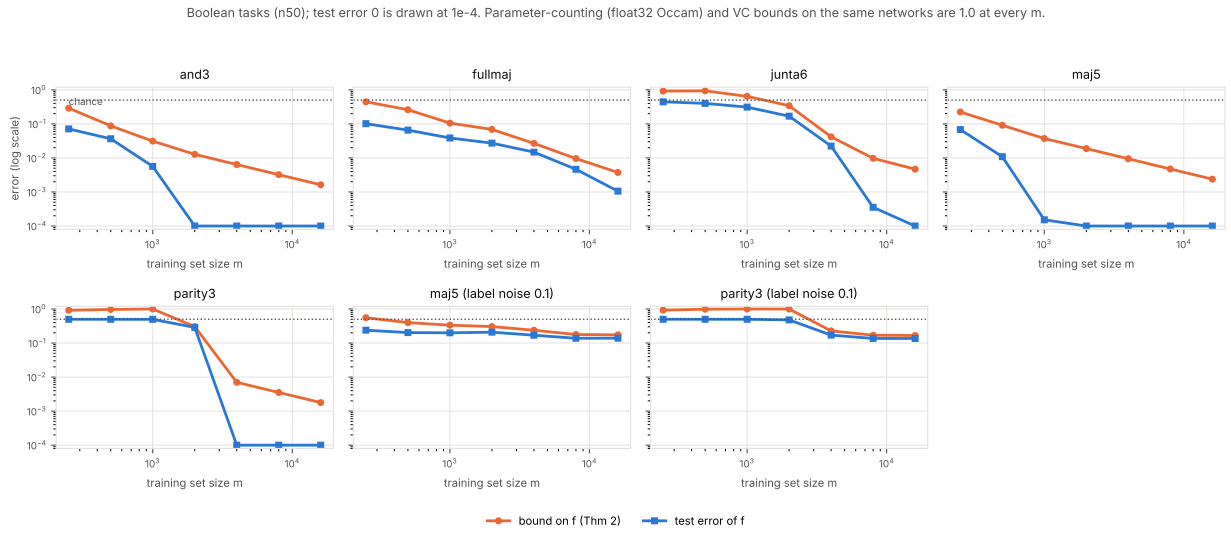


Figure 1: Boolean tasks, $n = 50$: test error of the trained network f (blue) and the certified upper bound on it from Theorem 2 (orange), averaged over 3 seeds, versus the number of training examples. The dotted line is chance.

5.2 Distribution-free bounds by exact equivalence

With $n = 20$ the whole input space has $2^{20} \approx 10^6$ points, so the disagreement between f and g can be computed exactly, and Corollary 3 turns compression into a bound that holds for *every* input distribution. Table 3 uses the same architecture and training protocol as before, with widths $(20, 512, 512, 1)$. Once a network has learned its target (test error below 10^{-5}), its compressed description agrees with it on all 2^{20} inputs, or on all but at most 8 of them. Appending the exception set E then costs at most about 150 bits, and the distribution-free bound is close to the uniform-distribution bound: 0.0065 against 0.0061 for `parity3` at $m = 4000$, 0.018 against 0.017 for `junta6`, and 0.024 against 0.008 for `fullmaj`, whose compressed descriptions disagree with f on 2–6 inputs. When f still errs on a small fraction of inputs (e.g. `maj5` at $m = 250$, with 560–3700 exceptions), the uniform-distribution bound is non-vacuous (0.12) but the distribution-free one is not: listing the exceptions costs far more bits than 250 training points can pay for. Listing E costs about $|E| \log_2(2^{20}e/|E|)$ bits (7,000–35,000 here), so Corollary 3 is informative only when that is small compared with m. This is a limitation of the exact-equivalence route, which has to describe f exactly, not of distribution-free bounds as such. For one-hidden-layer networks, the dithered transfer theorem (Section 5.4) gives distribution-free bounds without exact equivalence, by certifying f only at the inputs where its margins allow.

5.3 Images: bounds with unlabeled inputs

With unlabeled inputs from D_X, Theorem 2 applies to image classifiers directly; this is the setting of Bazinet et al. [3], and we include it to place the other results. The surrogate g is the init-relative, DCT-coded, quantized network of Section 4, fine-tuned with a rate penalty using training labels, f's outputs, or both. The disagreement is measured on the 10,000 test images without their labels, with a union bound over the K compression configurations tried for each f. Table 4 reports the best Occam bound on g itself and the best bound on f; each row is one trained network. On MNIST the bound on f is 0.31 for 100 hidden units (test error 2.3%, description 18k bits) and 0.37 for 256

(2.0%, 28k bits); on Fashion-MNIST it is 0.53 and 0.62. Parameter-counting and VC bounds are 1.0 for all of these networks. The disagreement term adds 0.03–0.11 to the Occam bound on g. Most of the rest is the cost of the description: g's training error is 2–14%, and 17–29 thousand bits against $m = 60000$ examples add another 0.24–0.39. Our compressor is simple and uses a data-independent prior, so these bounds are much weaker than those of Bazinet et al. [3], who report 1.8–3.5% for an MNIST CNN using surrogates trained on part of the training set: 1.8% for a surrogate retrained on a random subset and tested on the remaining training examples, 3.5% for a compressed surrogate with a data-dependent prior. Coding the first-layer update in the DCT basis instead of the pixel basis saves 22% of the bits (18.4k against 23.5k; bound 0.31 against 0.37). Every network selected the largest rate penalty on the grid, so a wider grid would probably help.

5.4 Images and Boolean tasks: bounds from training data alone

These are the bounds the question asks for in its strictest form: a deterministic bound on the error of the trained network itself, computed from the training set alone, with no validation data, no unlabeled data and no knowledge of the input distribution. The predictor certified is the stored weights of f evaluated in float64; all networks here have one hidden layer, and each row is one trained network (one seed).

Worst-case weight sets (Theorem 4) are hopeless for dense networks. On a 784–100–10 MNIST network (test error 2.4%) we quantized the first-layer update in the DCT basis and certified with per-unit, per-band ℓ_2 balls around the quantized network that contain f. At every step size the certified first-layer radius is 12–13 times the actual first-layer error $|Rx|$. Certifying 95% of the training points needs steps of 2% of the update's RMS, i.e. 6.1×10^5 bits for the first layer alone, ten times more than $m = 6 \times 10^4$ (`results/mnist/t3_worstcase_probe.txt`).

Dithered compression (Theorem 5 and Lemma 7). Table 5 reports the bound for one-hidden-layer networks trained in the standard way (Adam, 20 epochs). It is non-vacuous (below the chance level of 0.9) for narrow networks: 0.44 for $H = 8$ (test error 7.0%) and 0.55 for $H = 16$ (5.1%) on MNIST, and 0.66 for $H = 16$ on Fashion-MNIST. For these narrow networks the parameter-counting baselines are not entirely vacuous either: coding f exactly with 32 bits per parameter gives 0.94 and 0.99, and the VC bound with d equal to the number of parameters, an optimistic proxy, gives 0.71 and 0.87. It degrades with width: 0.70 at $H = 32$, 0.87 at $H = 64$, 0.97 at $H = 128$. Dithering does what it was designed to do. At comparable certified fractions it needs about 3 bits per first-layer coefficient instead of the 8–9 bits of worst-case sets (7.8 bits at 95% certified, 8.8 at 98%; `results/mnist/t3_worstcase_probe.txt`), because the certificate scales with the ℓ_2 aggregate of the noise rather than its ℓ_1 worst case. But at the measured 2.1–3.6 bits per first-layer coefficient (3.2–3.6 for $H \leq 32$), $784H$ coefficients exceed $m = 6 \times 10^4$ bits once $H \gtrsim 25$, and the total grows linearly in H.

Three ablations on the 32-unit network: coding the first layer in the pixel basis instead of the DCT basis costs 37% more bits (bound 0.81 instead of 0.70); training with Gaussian weight noise of one or two times the initial RMS changes little (0.71, 0.69); a deterministic DCT initialization gives a shorter description (67k instead of 84k bits), but also a lower certified fraction (0.90 instead of 0.95), so the bound barely changes (0.691 against 0.704; one seed each).

Networks with sparse updates. The cost is driven by the number of first-layer coefficients that have to be described. Since exactly-zero coefficients are sent exactly (no dither, no noise, Section 4), a network whose update is sparse in the DCT basis is much cheaper. We trained 100- and 256-unit

networks normally, kept the largest 0.1–3% of the update's DCT coefficients and fine-tuned with that support fixed. This procedure is chosen to make the bound small: it raises the test error (Table 6), and the deployed first-layer weights $W_0 + \Delta$ are still dense, since the update is sparse only in the DCT basis and relative to the free initialization. The bound is on the network that the procedure outputs. Table 6 shows the result. A standard-width MNIST network (100 hidden units) with 1% of its update coefficients nonzero has test error 4.6% and a training-data-only bound of 0.31. Keeping 0.3% gives 7.2% and 0.30, and 256-unit networks reach 0.40. The description is then 11–32 thousand bits. Each kept first-layer coefficient costs 5–8 bits for its value and 6–9 bits for its position, and the output layer takes 20–76% of the total. The right baseline for such a network is to code it exactly, with the same support code and 32 bits per stored parameter. That is already non-vacuous (0.62 for the 1% network, 0.55 for 0.3%), and dithering halves it (Table 6). For the output layer the search always picked the largest step multiplier on its grid, so a wider grid would probably help a little.

Boolean networks: a bound for every input distribution. Table 7 applies the same theorem to one-hidden-layer networks on Boolean tasks. Since it uses only the training sample, the bound holds whatever distribution generated the inputs. The bounds are 0.06–0.33 for networks with 16 or 32 hidden units at $m = 16000$ (0.23 at $m = 4000$). These networks have only 353–1665 parameters, so the parameter-counting baselines are not vacuous (32-bit Occam 0.39–0.90, VC 0.35–0.64); the dithered bound is 2–5 times below the VC bound. With the uniform distribution known, Theorem 2 on the same networks is 30–150 times tighter (0.0015–0.007). The gap is the price of a weight-space certificate against function-space compression: the dithered code needs 1500–9100 bits where a truth table or threshold function needs 30–100. Knowledge of the input distribution is not what closes it. For the $n = 20$ networks, Corollary 3 (exact equivalence with all 2^{20} inputs, no exceptions here) already gives distribution-free bounds of 0.0016–0.0076 on the same networks.

5.5 Initialization

Coding relative to the initialization. Table 8 trains the same 784–256–10 network from three initialization scales (multiples of the default initialization) and from a deterministic DCT initialization, one seed each, and codes the update either relative to the (free) initialization or relative to zero, with the unlabeled-data codec of Section 5.3. Coding relative to the initialization is always cheaper, and the saving grows with the scale: 2% at scale 0.3 (30.8k against 31.5k bits), 6% at scale 1 (27.7k against 29.3k) and 17% at scale 3 (24.7k against 29.6k). The saving is modest because g is fine-tuned through the quantizer with a rate penalty, so it need not stay close to f in weight space; coded relative to zero, it can shrink the initial weights instead of describing them. Relative to the initialization, the description gets shorter as the scale grows (24.7k bits at scale 3), which fits the picture of lazier training at larger scale, though we have not tested that explanation. The deterministic DCT initialization does worse than the seeded random one at scale 1 (31.2k against 27.7k bits). The bounds on f are 0.35–0.41 at test errors of 1.9–2.2%. Every row selected the largest rate penalty on the grid, so a wider grid would probably shorten all of these descriptions. Table 5 makes the same comparison for the dithered training-data-only code of a 32-unit network: coding relative to zero costs 84,269 bits against 83,976, a difference of 0.3%. The comparison is not exactly like for like, since the step search selected different configurations for the two references ($p_u = 0$, $v_m = 4$ relative to the initialization; $p_u = 0.5$, $v_m = 1$ relative to zero). The first-layer weights in fact cost slightly less relative to zero (80,290 bits against 80,839), and the output layer (3,315 against 2,598 bits) and the step fields more. We have two hypotheses for why the reference matters so little here, neither tested separately. First, Adam moves the first-layer weights far from

their initial values, so the update is not small compared with the initialization. Second, in the dithered code the steps in the high-frequency DCT bands are coarse, because natural images have little energy there, so describing the random initial weights in those bands costs almost nothing. In the dithered code the deterministic DCT initialization gave a shorter description (67k bits against 84k), but the bound-minimizing code certifies fewer training points (0.90 against 0.95), so the bound barely moves (0.691 against 0.704; one seed each). At a matched certified fraction the difference corresponds to about 3k bits (4%). Together with the unlabeled-data codec, where it does worse, this gives no evidence that a deterministic initialization helps.

The init-kernel codec in the lazy and rich regimes. Table 9 compares, on the same Boolean networks ($n = 30$, one hidden layer of 512 units), the init-kernel codec of Proposition 9 with function-space distillation. The lazy regime uses initialization scale 2 and SGD with momentum 0.9 (learning rate 0.02); the parameters then move by 9–18% of their initial norm, and f agrees with its linearization at θ_0 on 97–99.9% of inputs for the majority targets but only on 83–87% for `parity2`. The rich regime uses scale 0.1 and Adam; the parameters move by 3–5 times their initial norm and the linearization is poor. The init-kernel codec describes $\theta_0 + P_r\Delta$, where P_r projects onto the top-r eigenvectors of the uniform-input empirical kernel at θ_0. That kernel is estimated from 4000 uniform inputs, so it is free given θ_0. The candidate ranks are $r \in \{8, 16, 31, 64, 128, 256, 466, 1024, 2048\}$, which include the dimensions of the low-degree Walsh spaces $1 + n = 31$ and $1 + n + \binom{n}{2} = 466$. Each row is one network (one seed).

The results match the picture of Section 3.2.

- In the lazy regime, for the majority targets the selected rank is $r = 31$, the dimension of the functions of degree at most 1, and the bounds are 0.05–0.25. The estimated bound drops sharply from $r = 16$ to $r = 31$ (0.67 to 0.16 for `maj5` at $m = 1000$) and then rises slowly (0.18 at $r = 64$), as expected if the top 31 eigenvectors of the initial kernel span the degree-≤ 1 functions (the majority targets are signs of such functions). For the degree-2 target `parity2` the selected rank is $466 = 1 + n + \binom{n}{2}$, but the minimum is shallow (0.68, 0.59, 0.66 at $r = 256, 466, 1024$ for $m = 4000$; a tie with $r = 256$ at $m = 1000$), so the grid does not identify the rank sharply there. Describing 466 coefficients costs about 1500 bits. The bound is 0.59 at $m = 4000$: 0.14 is the disagreement between f and g, and 0.45 the Occam term for g, whose training error is 11.8% (with no training errors the Occam term would be 0.23). The projected network is thus a poor surrogate for `parity2`, as the 13% disagreement between f and its linearization already suggests. The distilled junta needs 20 bits and gets 0.006. If the pattern seen for degree 1 holds for degree k, the codec's cost grows like $\binom{n}{k}$, the n^k sample complexity of kernel methods appearing as description length; with one n and one degree-2 target this is a heuristic, not a measurement.
- In the rich regime the parameters move by 3.3–4.7 times their initial norm and the codec is weaker, though it does not always fail: 0.29–0.30 for `maj5` ($r = 31$ is selected for `maj5` at both m and for `fullmaj` at $m = 4000$), 0.50–0.82 for `fullmaj` and 0.77–1.0 for `parity2`.
- Function-space distillation does better in both regimes. It describes the function, not the update, so how far training moved from the initialization does not matter to it.

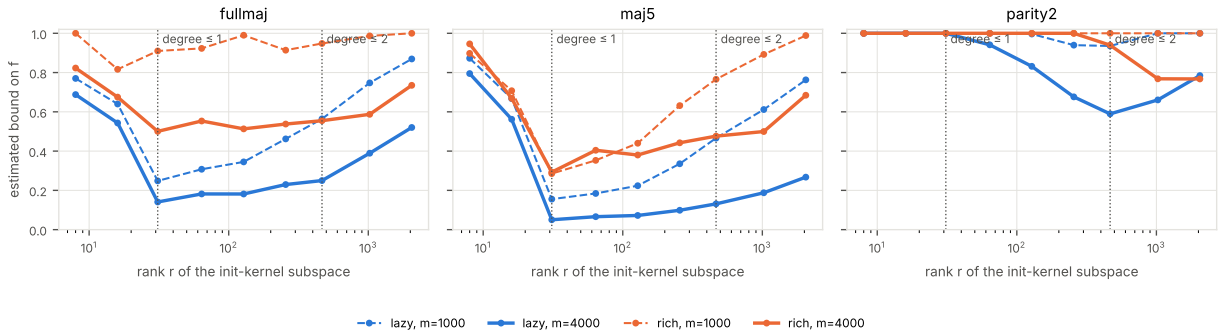


Figure 2: Init-kernel codec: estimated bound on f (Occam term plus disagreement on a selection pool) as a function of the rank r of the subspace spanned by the top eigenvectors of the initial kernel, for networks trained in the lazy (blue) and rich (orange) regimes. Dotted lines: dimensions of the Walsh spaces of degree at most 1 $(1+n)$ and at most 2 $(1+n+\binom{n}{2})$.

6 Discussion

When is the reduction tight? Optimizing Theorem 2 over descriptions gives, up to estimation error,

$$\mathrm{err}(f) \;\lesssim\; \min_{\sigma}\left[\mathrm{err}(g_\sigma) + \Pr_{D_X}[f \neq g_\sigma] + \frac{L(\sigma)\ln 2}{m}\right] \;\leq\; \mathrm{err}(f) + 2\epsilon + \frac{\ln 2}{m}\,K_\epsilon(f),$$

where $K_\epsilon(f) = \min\{L(\sigma) : \Pr_{D_X}[f \neq g_\sigma] \leq \epsilon\}$ is the ϵ-approximate description length of f under D_X (the first inequality uses $\mathrm{err}(g) \leq \mathrm{err}(f) + \epsilon$; the rough form uses the realizable rate $L\ln 2/m$, whereas the binomial inversion adds a term of order $\sqrt{\mathrm{err}(g)L/m}$ when g has training errors, so it is optimistic there). So the reduction is as good as f's *lossy* compressibility under the input distribution. Lossy is essential. A network that interpolates noisy labels differs from the clean target only near the training points, a set of small D_X-measure, so it can have small K_ϵ. In our label-noise experiments the bound on such interpolating networks is 0.17 against a test error of 0.14 (Section 5.1). The bound fails, correctly, when f is far from every short program; this includes the case of a good network for an incompressible target. Generalization can also be reduced to compression in the other sense, that of the sample [25], with the training procedure as decoder; the bound is useful only when the procedure, run on a small part of the training set, returns f or a function close to it. (Moran and Yehudayoff [30] show that every class of VC dimension d has a sample compression scheme of size $2^{O(d)}$, but its decoder is a majority vote of a learner for the class, not a given training procedure, and both results assume the labels come from the class.) This is a second description language for the same reduction, and its decoder can use the initialization for free.

Finding the description. The bound holds for whatever search produced g, provided the code is fixed in advance and the certification inputs are used once; compute affects only how tight it is. In the worst case the search is hard: if pseudorandom functions exist, no polynomial-time algorithm with query access to f finds an efficient g that agrees with f on uniformly random inputs noticeably better than chance, although f has a short description (the classical argument that small circuits cannot be learned from membership queries). Networks that gradient descent produces on learnable tasks need not be worst cases, and a companion note in the repository (`search/README.md`) measures the search directly. On the junta and threshold targets an influence-guided search recovers the target on every network. On the signs of random 4- and 8-unit ReLU networks, single runs of

distillation, unit subsampling and greedy selection stop at 1.2–1.9 times the bound the true target would give, and the best of 32 random restarts of a small student, trained longer and quantized more finely, reaches 0.96–1.11 times. Subsampling hidden units in the manner of Maurey's lemma works for mean-field networks, whose path norm does not grow with width, and not for standard ones. When retraining is possible, a network trained on a fixed half of the training set, tested on the other half and compared with f on unlabeled inputs, gives a bound with no search at all: 0.042 for a 784-1024-10 MNIST network with test error 1.55%, against 0.255 through compression. This is a special case of Kääriäinen [20] and the construction behind the tightest bounds of Bazinet et al. [3].

Provable closeness without inputs from D_X. The second term of Theorem 2 is where a validation-free bound has to work. With known or sampleable D_X it is free; with unlabeled inputs it is the method of Bazinet et al. [3]. With neither, the only information about D_X is the training inputs, which f depends on. The dithered transfer theorem resolves this by making the certificate a function of the description alone. It costs a weight-space description of f precise enough for f's margins, and the cost grows with the number of weights that matter at typical inputs. Functional simplicity of f (e.g. f is close to a 5-junta) is invisible to it, unlike to Theorem 2. Closing this gap is the main open problem the experiments point to. A certificate would have to establish $f(x) = g(x)$ at a point from a description of g plus a short proof about f, as in the compact-proofs programme of Gross et al. [14], rather than from weight-space proximity.

The initialization. The worry that the random initialization must be described is unfounded at the level of validity (Lemma 8). Any data-independent randomness can be used by the code for free, and a seed costs 32 bits anyway. What remains is a question about the update. (i) Function-space codecs (our Boolean truth tables and distilled students) never refer to f's initialization, so it cannot cost anything. (ii) Weight-space codecs, which training-data-only bounds need, code the update relative to θ_0. Weights that receive no gradient (first-layer weights on pixels that are zero in every training image) have update exactly zero and cost almost nothing, whereas coding them relative to zero costs their full entropy. (iii) Exact zero initialization is a degenerate point for ReLU MLPs (all hidden units are dead and only the output bias receives a gradient), so it does not train. A deterministic structured initialization trains and costs nothing, but a seeded random one also costs nothing. (iv) From a typical initialization, wide networks in the mean-field scaling (or with a centred initialization) trained by gradient descent approach a deterministic limit, so the initialization only adds fluctuations of order $\text{width}^{-1/2}$. In the NTK scaling the trained function is not deterministic: it is the kernel regression plus the initial function, an $O(1)$ Gaussian-process sample. A function-space codec absorbs such fluctuations into the disagreement term; a weight-space codec must pay for them or certify through them. In the linearized regime the free eigenbasis of the initial kernel gives a codec (Proposition 9) whose cost, heuristically, grows like the n^k sample complexity of kernel methods on degree-k targets; we observed the predicted rank $1 + n$ for majority targets (signs of degree-1 functions) at one value of n. A network that is close to a short junta is short to describe in function space whichever regime trained it: in our Boolean runs the lazily trained networks distil into the same 20–51-bit juntas and threshold functions as the richly trained ones, with bounds of 0.006–0.08 against 0.005–0.04 (the gap follows the lazy networks' larger test errors). The regime mattered only for the weight-space init-kernel codec, which did worse in the rich regime.

Limitations. The training-data-only bound is proved for one-hidden-layer networks. The same argument extends layer by layer (pre-activation errors stay sub-Gaussian, though no longer independent across units), but we have not carried it out. Our codecs and step grids were designed with

the datasets in mind, as in all of this literature: strictly, the code must be fixed before the sample is drawn. The Boolean experiments use small input dimension and synthetic targets. On images we bound MLPs, not convolutional networks, and our unlabeled-data bounds are far from the best published ones [3], which retrain a surrogate on a random part of the training set and test it on the rest, or use data-dependent priors and stronger compressors.

References

[1] Sanjeev Arora, Rong Ge, Behnam Neyshabur, and Yi Zhang. Stronger generalization bounds for deep nets via a compression approach. In *International Conference on Machine Learning (ICML)*, 2018.

[2] Peter L. Bartlett, Nick Harvey, Christopher Liaw, and Abbas Mehrabian. Nearly-tight VC-dimension and pseudodimension bounds for piecewise linear neural networks. *Journal of Machine Learning Research*, 20(63):1–17, 2019.

[3] Mathieu Bazinet, Valentina Zantedeschi, and Pascal Germain. Bound to disagree: Generalization bounds via certifiable surrogates. In *Conference on Uncertainty in Artificial Intelligence (UAI)*, volume 337 of *Proceedings of Machine Learning Research*, pages 491–520, 2026. arXiv:2602.23128. Code: https://github.com/GRAAL-Research/Bound-to-Disagree.

[4] Felix Biggs and Benjamin Guedj. On margins and derandomisation in PAC-Bayes. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2022.

[5] Felix Biggs and Benjamin Guedj. Non-vacuous generalisation bounds for shallow neural networks. In *International Conference on Machine Learning (ICML)*, 2022.

[6] Gilles Blanchard and François Fleuret. Occam's hammer. In *Proceedings of the Conference on Learning Theory (COLT)*, 2007.

[7] Anselm Blumer, Andrzej Ehrenfeucht, David Haussler, and Manfred K. Warmuth. Occam's razor. *Information Processing Letters*, 24(6):377–380, 1987.

[8] Lénaïc Chizat, Edouard Oyallon, and Francis Bach. On lazy training in differentiable programming. In *Advances in Neural Information Processing Systems 32*, 2019.

[9] Claude (Anthropic). Gradient descent as greedy growth of a shared circuit, 2026. Research note prompted by Kaarel Hänni, https://github.com/kaarelh/AI-works/tree/main/gd-circuit-growth; see its distillation certificates.

[10] Amit Daniely and Elad Granot. Generalization bounds for neural networks via approximate description length. In *Advances in Neural Information Processing Systems 32*, 2019.

[11] Gintare Karolina Dziugaite and Daniel M. Roy. Computing nonvacuous generalization bounds for deep (stochastic) neural networks with many more parameters than training data. In *Uncertainty in Artificial Intelligence (UAI)*, 2017.

[12] Gintare Karolina Dziugaite, Kyle Hsu, Waseem Gharbieh, Gabriel Arpino, and Daniel M. Roy. On the role of data in PAC-Bayes bounds. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2021.

[13] Saurabh Garg, Sivaraman Balakrishnan, J. Zico Kolter, and Zachary C. Lipton. RATT: Leveraging unlabeled data to guarantee generalization. In *International Conference on Machine Learning (ICML)*, 2021.

[14] Jason Gross, Rajashree Agrawal, Thomas Kwa, Euan Ong, Chun Hei Yip, Alex Gibson, Soufiane Noubir, and Lawrence Chan. Compact proofs of model performance via mechanistic interpretability, 2024. arXiv:2406.11779.

[15] Marton Havasi, Robert Peharz, and José Miguel Hernández-Lobato. Minimal random code learning: getting bits back from compressed model parameters. In *International Conference on Learning Representations (ICLR)*, 2019.

[16] Geoffrey E. Hinton and Drew van Camp. Keeping the neural networks simple by minimizing the description length of the weights. In *Proceedings of the Conference on Computational Learning Theory (COLT)*, 1993.

[17] Sepp Hochreiter and Jürgen Schmidhuber. Flat minima. *Neural Computation*, 9(1):1–42, 1997.

[18] Daniel Hsu, Ziwei Ji, Matus Telgarsky, and Lan Wang. Generalization bounds via distillation. In *International Conference on Learning Representations (ICLR)*, 2021.

[19] Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: convergence and generalization in neural networks. In *Advances in Neural Information Processing Systems 31*, 2018.

[20] Matti Kääriäinen. Generalization error bounds using unlabeled data. In *Learning Theory: 18th Annual Conference on Learning Theory (COLT)*, volume 3559 of *Lecture Notes in Computer Science*, pages 127–142. Springer, 2005.

[21] John Langford. Tutorial on practical prediction theory for classification. *Journal of Machine Learning Research*, 6:273–306, 2005.

[22] John Langford and Rich Caruana. (Not) bounding the true error. In *Advances in Neural Information Processing Systems 14*, 2002.

[23] John Langford and John Shawe-Taylor. PAC-Bayes & margins. In *Advances in Neural Information Processing Systems 15*, 2003.

[24] Chunyuan Li, Heerad Farkhoor, Rosanne Liu, and Jason Yosinski. Measuring the intrinsic dimension of objective landscapes. In *International Conference on Learning Representations (ICLR)*, 2018.

[25] Nick Littlestone and Manfred K. Warmuth. Relating data compression and learnability. Technical report, University of California, Santa Cruz, 1986.

[26] Sanae Lotfi, Marc Finzi, Sanyam Kapoor, Andres Potapczynski, Micah Goldblum, and Andrew Gordon Wilson. PAC-Bayes compression bounds so tight that they can explain generalization. In *Advances in Neural Information Processing Systems 35*, pages 31459–31473, 2022.

[27] Sanae Lotfi, Marc Finzi, Yilun Kuang, Tim G. J. Rudner, Micah Goldblum, and Andrew Gordon Wilson. Non-vacuous generalization bounds for large language models. In *International Conference on Machine Learning (ICML)*, 2024.

[28] David A. McAllester. PAC-Bayesian model averaging. In *Proceedings of the Conference on Computational Learning Theory (COLT)*, 1999.

[29] Song Mei, Andrea Montanari, and Phan-Minh Nguyen. A mean field view of the landscape of two-layer neural networks. *Proceedings of the National Academy of Sciences*, 115(33): E7665–E7671, 2018.

[30] Shay Moran and Amir Yehudayoff. Sample compression schemes for VC classes. *Journal of the ACM*, 63(3), 2016.

[31] Vaishnavh Nagarajan and J. Zico Kolter. Deterministic PAC-Bayesian generalization bounds for deep networks via generalizing noise-resilience. In *International Conference on Learning Representations (ICLR)*, 2019.

[32] Jeffrey Negrea, Gintare Karolina Dziugaite, and Daniel M. Roy. In defense of uniform convergence: generalization via derandomization with an application to interpolating predictors. In *International Conference on Machine Learning (ICML)*, 2020.

[33] Behnam Neyshabur, Srinadh Bhojanapalli, and Nathan Srebro. A PAC-Bayesian approach to spectrally-normalized margin bounds for neural networks. In *International Conference on Learning Representations (ICLR)*, 2018.

[34] María Pérez-Ortiz, Omar Rivasplata, John Shawe-Taylor, and Csaba Szepesvári. Tighter risk certificates for neural networks. *Journal of Machine Learning Research*, 22(227):1–40, 2021.

[35] Leonard Schuchman. Dither signals and their effect on quantization noise. *IEEE Transactions on Communication Technology*, 12(4):162–165, 1964.

[36] Milad Sefidgaran, Amin Gohari, Gaël Richard, and Umut Şimşekli. Rate-distortion theoretic generalization bounds for stochastic learning algorithms. In *Proceedings of the Conference on Learning Theory (COLT)*, 2022.

[37] Taiji Suzuki, Hiroshi Abe, and Tomoaki Nishimura. Compression based bound for non-compressed network: unified generalization error analysis of large compressible deep neural network. In *International Conference on Learning Representations (ICLR)*, 2020.

[38] Guillermo Valle-Pérez, Chico Q. Camargo, and Ard A. Louis. Deep learning generalizes because the parameter-function map is biased towards simple functions. In *International Conference on Learning Representations (ICLR)*, 2019.

[39] Colin Wei and Tengyu Ma. Improved sample complexities for deep networks and robust classification via an all-layer margin. In *International Conference on Learning Representations (ICLR)*, 2020.

[40] Kaidi Xu, Zhouxing Shi, Huan Zhang, Yihan Wang, Kai-Wei Chang, Minlie Huang, Bhavya Kailkhura, Xue Lin, and Cho-Jui Hsieh. Automatic perturbation analysis for scalable certified robustness and beyond. In *Advances in Neural Information Processing Systems 33*, 2020.

[41] Jiawei Zhao, Florian Schäfer, and Anima Anandkumar. ZerO initialization: initializing neural networks with only zeros and ones. *Transactions on Machine Learning Research*, 2022.

[42] Hattie Zhou, Janice Lan, Rosanne Liu, and Jason Yosinski. Deconstructing lottery tickets: zeros, signs, and the supermask. In *Advances in Neural Information Processing Systems 32*, 2019.

[43] Wenda Zhou, Victor Veitch, Morgane Austern, Ryan P. Adams, and Peter Orbanz. Non-vacuous generalization bounds at the ImageNet scale: a PAC-Bayesian compression approach. In *International Conference on Learning Representations (ICLR)*, 2019.

A Experimental details

Boolean tasks. Relevant coordinates are a fixed random subset (seed 1000). Training inputs, labels and label noise use a seed determined by the run's seed and m. The network has PyTorch's default initialization (uniform on $\pm 1/\sqrt{\text{fan-in}}$ for weights and biases), drawn from an explicit generator seeded by the run's seed. Training stops at zero training error and mean training loss at most 0.01, or after 4×10^4 steps. The influence estimates use 2000 inputs per coordinate. Students are distilled for 2000 Adam steps (batch 256, learning rate 3×10^{-3}, cosine schedule) on a pool of 10^5 uniform inputs labelled by f. Quantization-aware fine-tuning runs 500 steps at learning rate 10^{-3} for each of three steps $\{0.5, 0.25, 0.125\}\times$RMS per layer. Candidate selection uses 3×10^4 further uniform inputs. For $n = 50$, the certified disagreement uses 2×10^5 fresh inputs. In the runs that record a `cert_seed` (36 of 147), these come from a generator seeded with OS entropy after g is fixed. The other 111 runs, made before that change, used a fixed seed $(4242 + 1000 \cdot \text{seed} + m)$ that was independent of g's construction but had also been used in development runs. They are covered by the design charge below, which takes the disagreement term at $\delta_2/16$. For $n = 20$ it is computed on all 2^{20} inputs. The predictor f whose error is bounded is the float64 evaluation of the trained weights.

Images. Pixels are scaled to $[0, 1]$. One-hidden-layer networks are trained with Adam (learning rate 10^{-3}, batch 128) for 20 epochs. Weight-noise training adds, at every step, Gaussian noise of the stated multiple of each tensor's initial RMS before the forward pass, and removes it before the update. Sparse-update networks are trained densely for 10 epochs in the parameterization $W = W_0 + AB^\top$. The largest coefficients of A are then kept and the network is fine-tuned for 5 epochs at learning rate 5×10^{-4} with the support fixed. The dither step grid has 162 points: a global scale $\gamma \in \{0.25, 0.5, 1, 2, 4, 8\}$ (first-layer steps $\gamma \cdot 10^{-2}$ times the band and unit factors), band exponents $p_b \in \{0.5, 0.75, 1\}$, unit exponents $p_u \in \{0, 0.5, 1\}$, and the hidden-bias and output-weight step $0.002\,\gamma\,v_m$ with $v_m \in \{1, 4, 16\}$ (the output bias has the fixed step 2^{-10}). The point is chosen on 6000 training points with a selection dither. (A first pass used a fixed $v_m = 1$; its results are kept on the working branch, in `results/mnist/*search_v1.jsonl`, and are not reported.)

Design charge. The codec families, step grids, band layout and certificate parameters were developed with exploratory runs on the same datasets. We therefore charge every description 4 bits, and take every disagreement term at confidence $\delta_2/16$. This covers a menu of up to 16 designs (`code/experiments/accounting.py`). It costs about $4 \ln 2/m$ in the Occam term, plus a slightly wider disagreement interval. On the reported means this is at most 0.025 on the Boolean grids (at $m = 100$) and 0.014 at $m = 250$, under 0.003 for the MNIST bounds that use unlabeled inputs, under 6×10^{-4} for the training-data-only Boolean bounds (under 2×10^{-4} at $m = 16000$) and under 10^{-4} for the training-data-only MNIST bounds.

B Provenance and verification

This work was produced by an AI system (Claude) in one session. The literature review (`literature.md`) was assembled by AI agents from web-search results and from the public repositories of Bazinet et al. [3] and Lotfi et al. [26]. Partway through, the session's web-search budget was exhausted, so some citations are marked there as unconfirmed. The theory and its implementation were checked by four independent AI referees, each with a different lens: the proof of Theorem 5 and Lemma 7; code against proof, including an independent reimplementation of the certificate; the remaining theorems and the experimental protocols; and attempts to break the bounds numerically. Each claimed defect was then checked by an independent adjudicator. A second round used three further referees (a skeptical reading of the paper, the code added since the first round, and the claims checked against the raw result files), again with one adjudicator per claimed defect. Neither round found an error invalidating a theorem or a bound. The defects found (statement scope, notation, test coverage, bookkeeping of selection effects in the first round; overclaims, descriptions of prior work and missing baselines in the second) were fixed and are listed in `verification/README.md`. No human has checked the proofs or the code.

task	m	test err. f	bound on f	bits of g
and3	500	0.0366	0.0875	32
and3	1000	0.0056	0.0312	32
and3	2000	0.0000	0.0128	32
and3	4000	0.0000	0.0064	32
and3	16000	0.0000	0.0016	32
fullmaj	500	0.0656	0.2609	103
fullmaj	1000	0.0386	0.1056	77
fullmaj	2000	0.0272	0.0688	94
fullmaj	4000	0.0148	0.0267	60
fullmaj	16000	0.0011	0.0037	48
junta6	500	0.4005	0.9364	61
junta6	1000	0.3111	0.6451	78
junta6	2000	0.1685	0.3422	102
junta6	4000	0.0223	0.0418	102
junta6	16000	0.0000	0.0047	102
maj5	500	0.0109	0.0910	54
maj5	1000	0.0002	0.0373	49
maj5	2000	0.0001	0.0188	49
maj5	4000	0.0000	0.0095	49
maj5	16000	0.0000	0.0024	49
parity3	500	0.4982	0.9695	87
parity3	1000	0.4964	1.0000	225
parity3	2000	0.2876	0.3043	35
parity3	4000	0.0000	0.0070	35
parity3	16000	0.0000	0.0018	35
maj5, noise 0.1	500	0.2028	0.3987	60
maj5, noise 0.1	1000	0.1997	0.3352	60
maj5, noise 0.1	2000	0.2088	0.3050	53
maj5, noise 0.1	4000	0.1696	0.2376	60
maj5, noise 0.1	16000	0.1390	0.1741	53
parity3, noise 0.1	500	0.4991	0.9848	155
parity3, noise 0.1	1000	0.4987	1.0000	218
parity3, noise 0.1	2000	0.4786	1.0000	228
parity3, noise 0.1	4000	0.1708	0.2275	35
parity3, noise 0.1	16000	0.1362	0.1665	35

Table 2: Boolean tasks, $n = 50$: test error of the trained network f (289,281 parameters) and the bound on it (Theorem 2), averaged over 3 seeds; 'bits' is the description length of the compressed g (including the 4-bit design charge). The parameter-counting and VC bounds for f are 1.0 in every row.

task	m	test err. f	bound on f	bits of g	any-D bound	$\lvert E\rvert$
`fullmaj`	100	0.1088	0.5449	90	1.0000	61359
`fullmaj`	250	0.0366	0.2307	71	1.0000	35489
`fullmaj`	500	0.0094	0.0763	46	1.0000	9810
`fullmaj`	1000	0.0010	0.0319	41	0.9986	1040
`fullmaj`	4000	0.0000	0.0078	41	0.0236	5
`junta6`	100	0.3811	0.8752	21	1.0000	294800
`junta6`	250	0.3067	0.7138	49	1.0000	233841
`junta6`	500	0.2206	0.5784	81	1.0000	185341
`junta6`	1000	0.0694	0.1662	93	1.0000	63981
`junta6`	4000	0.0000	0.0168	93	0.0184	0
`maj5`	100	0.0700	0.3870	51	1.0000	73405
`maj5`	250	0.0022	0.1225	42	1.0000	2257
`maj5`	500	0.0000	0.0621	42	0.1237	3
`maj5`	1000	0.0000	0.0315	42	0.0431	1
`maj5`	4000	0.0000	0.0080	42	0.0083	0
`parity3`	100	0.4933	0.9631	33	1.0000	300840
`parity3`	250	0.4507	0.9118	449	1.0000	255715
`parity3`	500	0.1834	0.2773	31	1.0000	190059
`parity3`	1000	0.0000	0.0242	31	0.0545	2
`parity3`	4000	0.0000	0.0061	31	0.0065	0

Table 3: Boolean tasks, $n = 20$: test error of the trained network f (273,921 parameters) and the bound on it (Theorem 2), averaged over 3 seeds; 'bits' is the description length of the compressed g (including the 4-bit design charge). The any-D bound (Corollary 3) holds for every input distribution; it patches g on the set E of inputs where $g \neq f$ (enumerated over all 2^{20} inputs). The parameter-counting and VC bounds for f are 1.0 in every row.

data	network	basis	test err. f	K	bound on g	bits	bound on f	bits	$\Pr[f \neq g]$
fashion	784-100-10	dct	0.1168	8	0.422	16594	0.532	16594	0.0997
fashion	784-256-10	dct	0.1106	8	0.512	28755	0.619	28755	0.0968
mnist	784-100-10	dct	0.0229	27	0.276	18371	0.313	18371	0.0302
mnist	784-100-10	id	0.0229	8	0.327	23522	0.370	23522	0.0352
mnist	784-256-10	dct	0.0197	27	0.339	27651	0.368	27651	0.0227

Table 4: Bounds with unlabeled inputs. 'Bound on g': Occam bound (Lemma 1) for the best compressed network among K configurations. 'Bound on f': Theorem 2, with $\Pr[f \neq g]$ estimated on the 10,000 test images without their labels, union-bounded over the K configurations. Parameter-counting and VC bounds for f are 1.0 in every row.

data	H	training	code	test err. f	bits	certified	bound on f	32-bit Occam	VC
mnist	8	standard	DCT	0.0697	23490	0.897	0.439	0.94	0.71
mnist	16	standard	DCT	0.0512	43658	0.919	0.553	0.99	0.87
mnist	32	standard	DCT	0.0323	83976	0.949	0.704	1.00	1.00
mnist	32	standard	DCT, DCT init	0.0331	67169	0.903	0.691	1.00	1.00
mnist	32	standard	DCT, ref. 0	0.0323	84269	0.950	0.704	1.00	1.00
mnist	32	standard	pixel	0.0323	114699	0.941	0.805	1.00	1.00
mnist	64	standard	DCT	0.0250	157051	0.964	0.871	1.00	1.00
mnist	128	standard	DCT	0.0213	223592	0.863	0.970	1.00	1.00
mnist	16	noise 1	DCT	0.0473	51751	0.938	0.573	0.99	0.87
mnist	32	noise 1	DCT	0.0320	81281	0.937	0.708	1.00	1.00
mnist	64	noise 1	DCT	0.0218	147536	0.960	0.858	1.00	1.00
mnist	100	noise 1	DCT	0.0210	221482	0.971	0.939	1.00	1.00
mnist	32	noise 2	DCT	0.0319	75306	0.932	0.691	1.00	1.00
mnist	100	noise 2	DCT	0.0199	212872	0.969	0.933	1.00	1.00
fashion	16	standard	DCT	0.1407	43032	0.826	0.659	1.00	0.95
fashion	32	standard	DCT	0.1305	77000	0.831	0.793	1.00	1.00
fashion	64	standard	DCT	0.1245	149239	0.833	0.926	1.00	1.00
fashion	32	noise 1	DCT	0.1305	78906	0.811	0.813	1.00	1.00

Table 5: Bounds on the original one-hidden-layer network f from the 60,000 training examples alone (Theorem 5 and Lemma 7): no validation set and no unlabeled data. 'Certified': fraction of training points with $\pi_g = 1$. 'Noise σ': f trained with Gaussian weight noise of σ times each tensor's initial RMS. Baselines for f: the Occam bound with 32 bits per parameter, and the VC bound with d equal to the number of parameters (optimistic, Section 4). One seed per row. Chance level is 0.9.

data	H	kept coeff.	test err. f	bits	certified	bound on f	exact code: bits	bound
mnist	100	3.0%	0.0333	32455	0.963	0.407	124318	0.797
mnist	100	1.0%	0.0462	17664	0.945	0.314	65754	0.618
mnist	100	0.3%	0.0722	11423	0.909	0.304	44864	0.553
mnist	256	0.3%	0.0441	23244	0.924	0.399	114452	0.792
mnist	256	0.1%	0.0647	21673	0.916	0.398	98696	0.770
fashion	100	1.0%	0.1693	20176	0.828	0.501	66862	0.746
fashion	100	0.3%	0.1926	13170	0.798	0.468	45307	0.680

Table 6: As Table 5, for networks whose first-layer update is exactly sparse in the DCT basis (trained, pruned to the stated fraction of coefficients, fine-tuned with the support fixed). The support is part of the description; only nonzero coefficients are dithered. The last two columns are the baseline for such a network: the Occam bound for coding f exactly, with the same support code and 32 bits per stored parameter. One seed per row.

					baselines		training data only		known D_X	
task	n	m	H	test err. f	32-bit Occam	VC	bits	bound	bits	bound
`maj5`	20	4000	16	0.0000	0.86	0.60	1529	0.234	38	0.007
`maj5`	20	16000	16	0.0000	0.39	0.35	1523	0.064	38	0.002
`parity3`	20	16000	16	0.0000	0.39	0.35	2009	0.084	31	0.002
`and3`	50	16000	16	0.0000	0.69	0.49	3791	0.152	32	0.002
`fullmaj`	50	16000	16	0.0000	0.69	0.49	3629	0.151	48	0.002
`junta6`	50	16000	32	0.0000	0.90	0.64	9132	0.327	102	0.005
`maj5`	50	16000	16	0.0000	0.69	0.49	2589	0.110	45	0.002
`maj5`, noise 1	50	16000	16	0.0000	0.69	0.49	2331	0.099	45	0.002
`maj5`	50	16000	32	0.0000	0.90	0.64	5468	0.213	45	0.002
`parity3`	50	16000	32	0.0000	0.90	0.64	7113	0.266	35	0.002

Table 7: Boolean tasks, one-hidden-layer networks, one seed each: the training-data-only bound (Theorem 5), which holds whatever the input distribution, against the bound that uses the known uniform input distribution (Theorem 2) on the same network. 'Noise σ': f trained with Gaussian weight noise of σ times each tensor's initial RMS. Baselines for f as in Table 5; with only 353–1665 parameters they are not vacuous here.

initialization	scale	code relative to	test err. f	bound on g	bits	bound on f	bits
deterministic DCT	1	initialization	0.0219	0.373	31243	0.405	31243
seeded random	0.3	initialization	0.0199	0.367	30794	0.397	30794
seeded random	0.3	zero	0.0199	0.375	31510	0.405	31510
seeded random	1	initialization	0.0197	0.339	27651	0.368	27651
seeded random	1	zero	0.0197	0.357	29274	0.386	29274
seeded random	3	initialization	0.0187	0.317	24654	0.346	24654
seeded random	3	zero	0.0187	0.361	29631	0.390	29631

Table 8: MNIST, 784-256-10: description length and bounds when the update is coded relative to the (free) initialization or relative to zero, for three initialization scales and a deterministic initialization. 'Scale' multiplies the default (PyTorch) initialization. One network per row; best of 8 compression configurations each, with the bound on f union-bounded over the 8.

task	regime	m	test err. f	move	lin. dis.	r	bits	init-kernel bound	distillation bound
`fullmaj`	lazy	1000	0.0387	0.11	0.015	31	205	0.253	0.080
`fullmaj`	rich	1000	0.0000	4.57	0.346	16	124	0.821	0.036
`fullmaj`	lazy	4000	0.0039	0.17	0.032	31	241	0.143	0.013
`fullmaj`	rich	4000	0.0000	4.71	0.277	31	196	0.502	0.009
`maj5`	lazy	1000	0.0010	0.09	0.002	31	208	0.158	0.033
`maj5`	rich	1000	0.0000	3.32	0.080	31	170	0.288	0.032
`maj5`	lazy	4000	0.0003	0.09	0.001	31	214	0.052	0.009
`maj5`	rich	4000	0.0000	3.98	0.197	31	213	0.297	0.008
`parity2`	lazy	1000	0.0181	0.17	0.168	466	1519	0.940	0.039
`parity2`	rich	1000	0.0000	3.58	0.434	2048	8230	1.000	0.020
`parity2`	lazy	4000	0.0003	0.18	0.133	466	1524	0.591	0.006
`parity2`	rich	4000	0.0000	3.53	0.410	2048	6094	0.772	0.005

Table 9: Boolean tasks ($n = 30$, one hidden layer of 512): the init-kernel codec of Proposition 9 (θ_0 plus the top-r eigenvectors of the uniform-input empirical NTK at θ_0, both free) against function-space distillation, on the same networks. 'Move': $\|\theta - \theta_0\|/\|\theta_0\|$; 'lin. dis.': disagreement between f and its linearization at θ_0. One seed per row.